МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Проверка условий

Студент
гр. БИН-26-1 _____ У. Т. Чан-Ван-Ю

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Управление кондиционером

Алгоритм решения первого задания представлен на блок-схеме (рисунок 1).

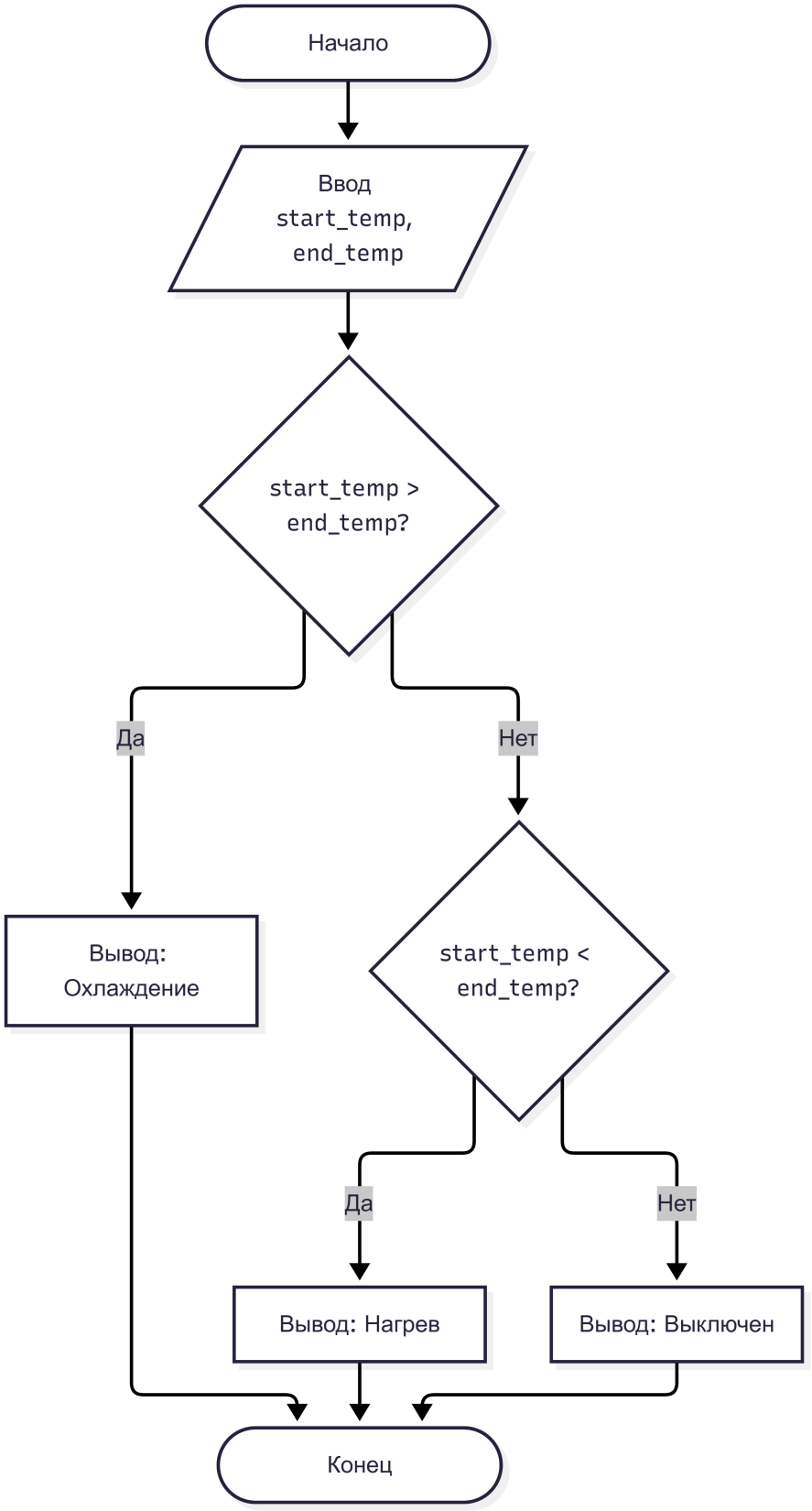


Рисунок 1 – Блок-схема алгоритма для первого задания

Первое задание моделирует работу системы управления климатом. Пользователь вводит два вещественных числа, представляющих текущую и желаемую температуры воздуха, каждое на отдельной строке. В зависимости от соотношения этих значений программа должна определить режим работы кондиционера: `Охлаждение`, если текущая температура превышает желаемую; `Нагрев`, если она ниже желаемой; и `Выключен`, если температуры совпадают.

Исходный код программы с результатами её выполнения приведён на рисунке 2.

```
1   start_temp = float(input())
2   end_temp = float(input())
3
4   if start_temp > end_temp:
5       print("Охлаждение")
6
7   elif start_temp < end_temp:
8       print("Нагрев")
9
10  else:
11      print("Выключен")
```

Рисунок 2 – Исходный код и результат выполнения первого задания

Для реализации алгоритма используется конструкция `if-elif-else`, позволяющая выбрать один из трёх взаимоисключающих вариантов. Ввод данных осуществляется функцией `input()` с последующим преобразованием в вещественный тип посредством `float()`. Сравнение температур производится операторами `>`, `<` и `==`, после чего соответствующий результат выводится на экран функцией `print()`.

1.2 Задание 2. Расчёт стоимости билета

Алгоритм решения второго задания представлен на блок-схеме (рисунок 3).

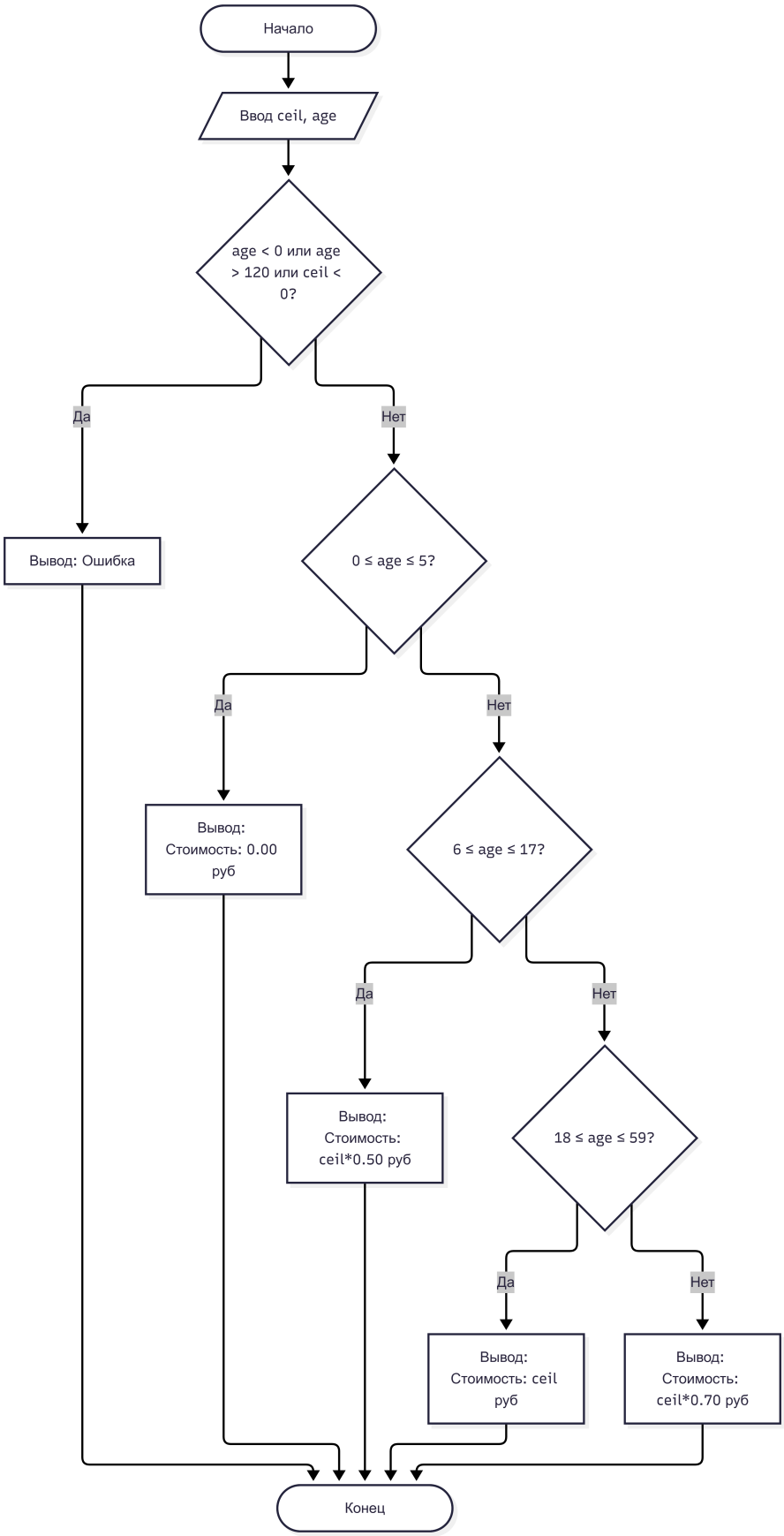


Рисунок 3 – Блок-схема алгоритма для второго задания

Во втором задании требуется рассчитать стоимость билета в зависимости от возраста пассажира. Пользователь вводит базовую стоимость (вещественное число) и возраст (целое

число) на отдельных строках. Программа должна проверить корректность входных данных: стоимость обязана быть положительной, а возраст — находиться в диапазоне от 0 до 120 лет включительно. При нарушении любого из этих условий выводится сообщение `Ошибка`. Для корректных данных цена рассчитывается согласно возрастной категории: дети до 5 лет — бесплатно, от 6 до 17 лет — 50% от базовой стоимости, от 18 до 59 лет — полная стоимость, от 60 до 120 лет — 70% от базовой стоимости.

Исходный код программы с результатами её выполнения приведён на рисунке 4.

```
ceil = float(input())
age = int(input())

if age > 120 or age < 0 or ceil < 0:
    print("Ошибка")

elif 0 <= age <= 5:
    print(f"Стоимость: {int(ceil * 0):.2f}")

elif 6 <= age <= 17:
    print(f"Стоимость: {int((ceil / 100) * 50):.2f}")

elif 18 <= age <= 59:
    print(f"Стоимость {int(ceil):.2f}")

elif 60 <= age <= 120:
    print(f"Стоимость {int((ceil/100) * 70):.2f}")
```

Рисунок 4 – Исходный код и результат выполнения второго задания

Проверка ограничений реализована в первом условном блоке `if`, объединяющем три условия через логический оператор `or`. Далее следует цепочка `elif`, в которой каждая ветвь соответствует своему возрастному диапазону. Для вычисления доли базовой стоимости применяются арифметические операции, а итоговый результат форматируется с двумя знаками после десятичной точки при помощи спецификатора `.2f` внутри f-строки.

1.3 Задание 3. Определение положения точки

Алгоритм решения третьего задания представлен на блок-схеме (рисунок 5).

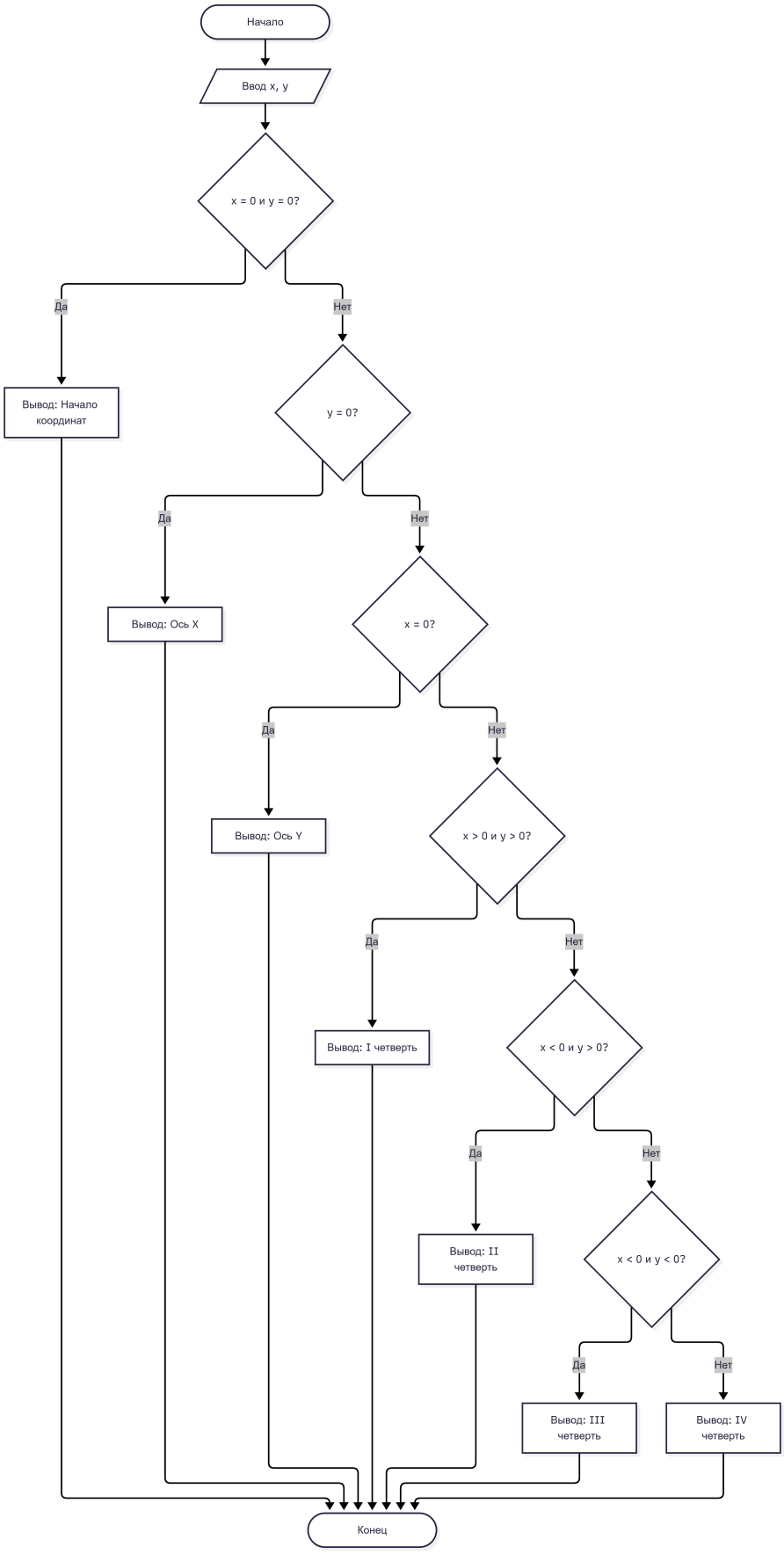


Рисунок 5 – Блок-схема алгоритма для третьего задания

Третье задание заключается в определении положения точки на координатной плоскости по её декартовым координатам x и y, вводимым пользователем как два вещественных

числа. Программа должна распознать один из семи возможных случаев: начало координат (обе координаты равны нулю), принадлежность оси `X` или оси `Y`, а также нахождение в одной из четырёх четвертей плоскости.

Исходный код программы с результатами её выполнения приведён на рисунке 6.

```
x = float(input())
y = float(input())

if x == 0 and y == 0:
    print("Начало координат")

elif y == 0 and x != 0:
    print("Ось X")

elif x == 0 and y != 0:
    print("Ось Y")

elif x > 0 and y > 0:
    print("I четверть")

elif x < 0 and y > 0:
    print("II четверть")

elif x < 0 and y < 0:
    print("III четверть")

elif x > 0 and y < 0:
    print("IV четверть")
```

Рисунок 6 – Исходный код и результат выполнения третьего задания

Алгоритм построен на последовательной проверке условий с использованием конструкции `if-elif-else`. Сначала обрабатываются особые случаи (равенство координат нулю), затем — принадлежность осям, и в завершение — определение четверти по знакам координат.

Сравнения выполняются операторами ==, !=, > и <, а для объединения условий применяется логический оператор and.

1.4 Задание 4. Проверка треугольника

Алгоритм решения четвёртого задания представлен на блок-схеме (рисунок 7).

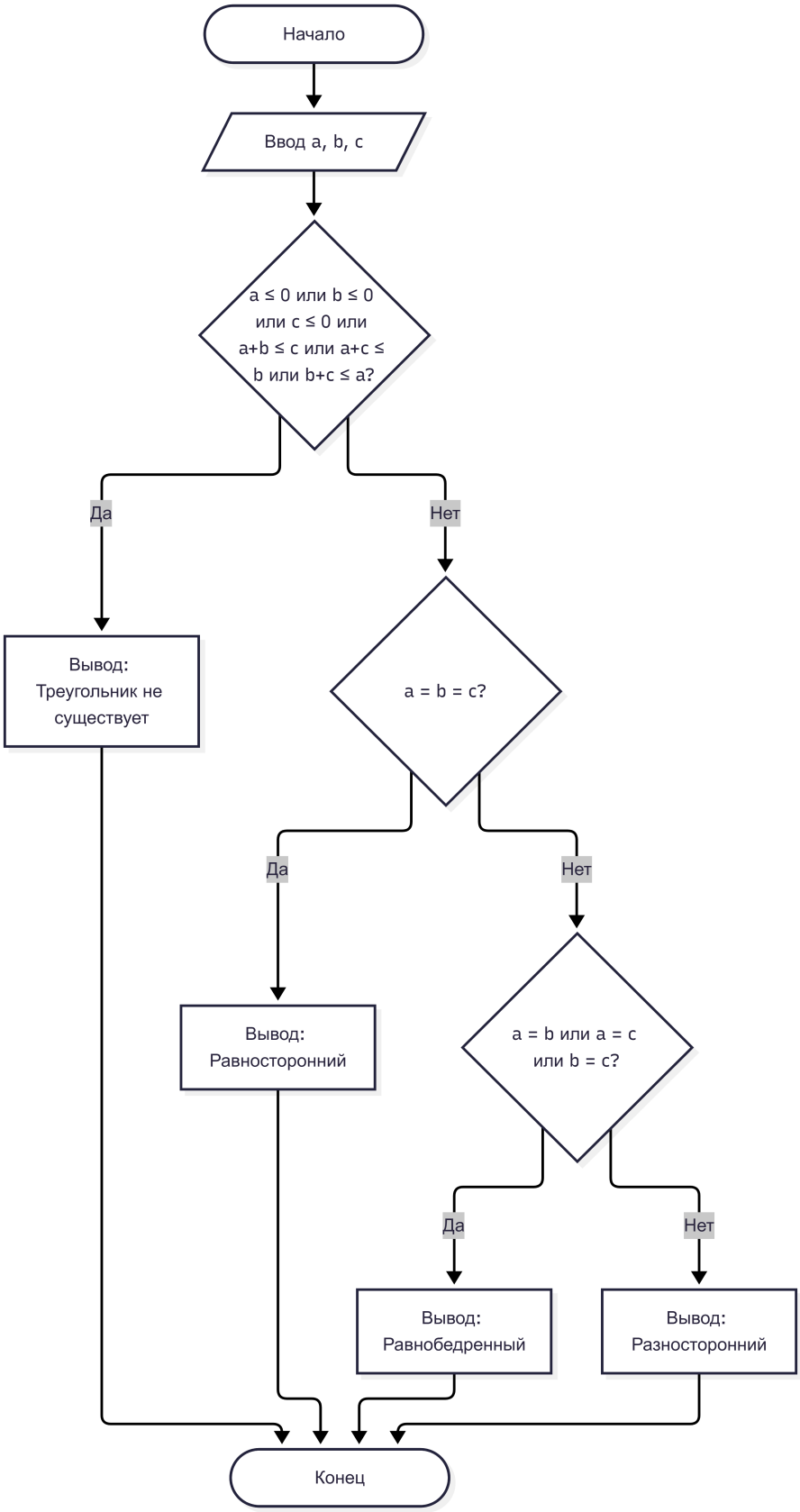


Рисунок 7 – Блок-схема алгоритма для четвёртого задания

Четвёртое задание требует определить тип треугольника по длинам его трёх сторон, вводимых пользователем как целые числа. Сначала программа проверяет существование треугольника: если хотя бы одна сторона неположительна или сумма каких-либо двух сторон не

превышает третью, выводится сообщение `Треугольник не существует`. Для существующего треугольника определяется его вид: `Равносторонний` (все три стороны равны), `Равнобедренный` (равны ровно две стороны) или `Разносторонний` (все стороны различны).

Исходный код программы с результатами её выполнения приведён на рисунке 8.

```
a = int(input())
b = int(input())
c = int(input())

if a <= 0 or b <= 0 or c <= 0:
    print("Треугольник не существует")

elif a == b == c:
    print("Равносторонний")

elif a == b or a == c or b == c:
    print("Равнобедренный")

else:
    print("Разносторонний")

```

Рисунок 8 – Исходный код и результат выполнения четвёртого задания

Проверка существования треугольника реализована через условие, объединяющее проверку на положительность сторон и неравенство треугольника. Для определения типа используется цепочка `if-elif-else`: сначала проверяется равенство всех трёх сторон, затем — наличие хотя бы одной пары равных сторон, и в противном случае треугольник признаётся разносторонним.

1.5 Задание 5. Определение високосного года

Алгоритм решения пятого задания представлен на блок-схеме (рисунок 9).

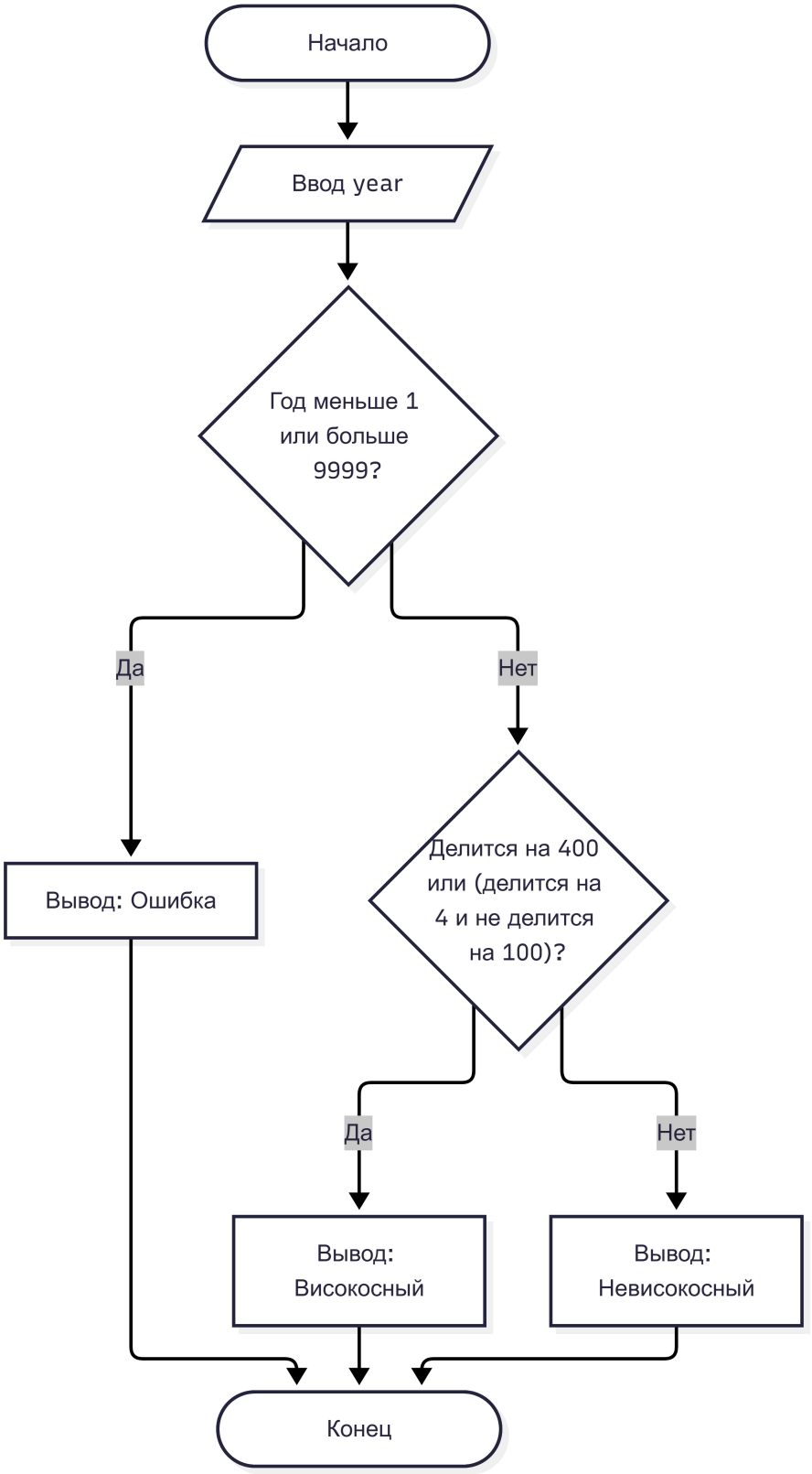


Рисунок 9 – Блок-схема алгоритма для пятого задания

В пятом задании необходимо определить, является ли введённый пользователем год високосным. Номер года — целое число; если оно выходит за пределы диапазона от 1 до 9999, программа выводит `Ошибка`. Для корректного значения применяется правило: год считается

високосным, если он делится на 400, либо одновременно делится на 4 и не делится на 100. В противном случае год признается невисокосным.

Исходный код программы с результатами её выполнения приведён на рисунке 10.

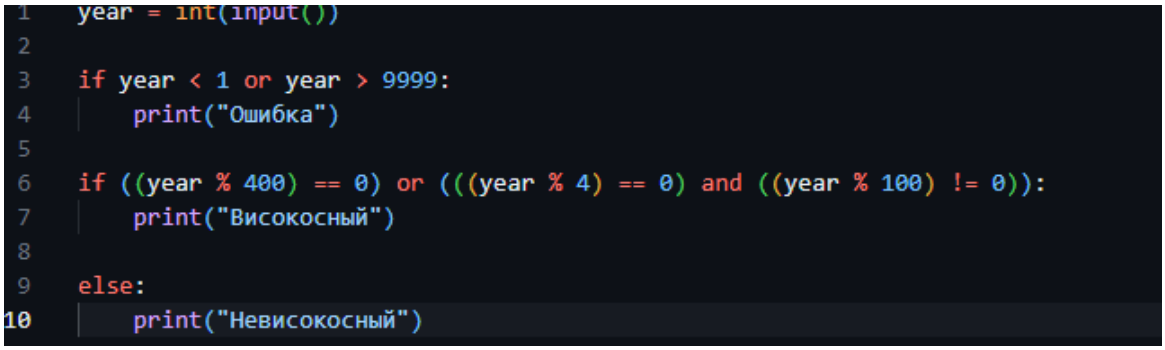


Рисунок 10 – Исходный код и результат выполнения пятого задания

Сначала выполняется проверка допустимого диапазона года. Далее применяется составное логическое условие с использованием операторов остатка от деления (`%`), а также логических операторов `or` и `and` для реализации правила определения високосного года. Результат выводится функцией `print()` в виде соответствующей строки.